

Problem 1

1. On the Gisette dataset, train decision trees of maximum depth $1, 2, \dots$ up to 12, for a total of 12 decision trees. Use the trained trees to predict the class labels on the training and test sets and obtain the training and test misclassification errors. Plot on the same graph the training and test misclassification errors vs tree depth as two separate curves. Report in a table the minimum test error and the tree depth for which the minimum was attained.

We do this in Python, and obtain a plot of the training and test misclassification errors vs tree depth in figure 1:

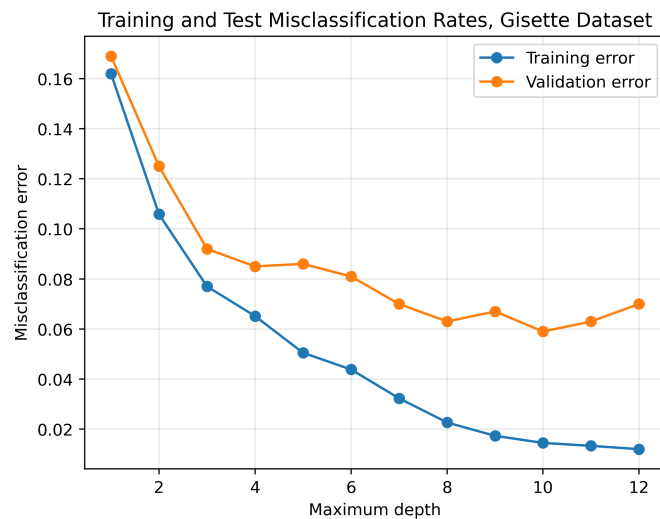


Figure 1: Misclassification Error vs Tree Depth, Training and Validation: Gisette dataset

The table reporting the minimum test error and the tree depth for which the minimum was attained are displayed in table 1.

Depth	Training Misclassification	Validation Misclassification
10	0.014500	0.059000

Table 1: Lowest misclassification error, Gisette Dataset

In general, the training error decreased as the depth increased. The validation error more or less followed this trajectory, but levelled off for deeper trees.

2. Repeat point 1.) on the satimage, madelon, hill-valley, and covtype datasets.

We can do this also, and obtain the four images shown in figure 2.

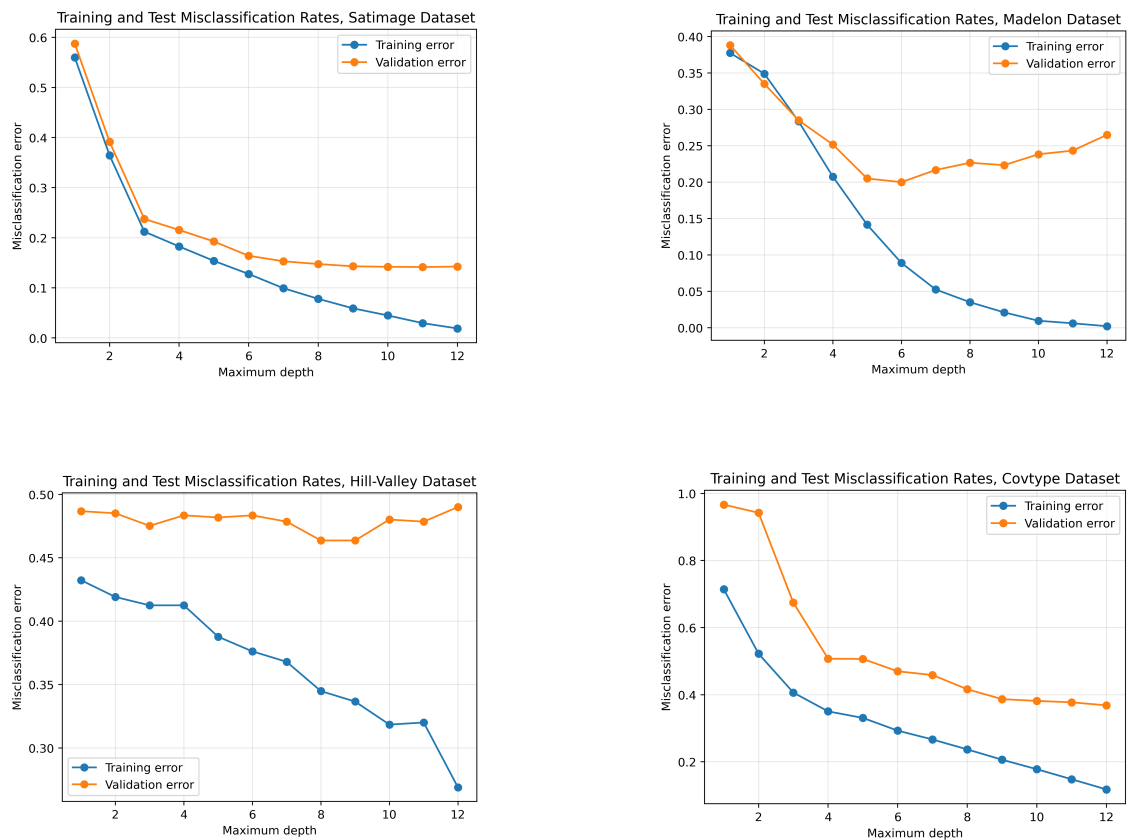


Figure 2: Training and Validation datasets misclassification error rates for four datasets

The table reporting the minimum test error and tree depth for the various datasets is shown in table 2

Dataset	Depth	Training Misclassification	Validation Misclassification
Satimage	11	0.029538	0.141500
Madelon	6	0.089000	0.200000
Hill-Valley	8	0.344884	0.463696
Covtype	12	0.117725	0.368371

Table 2: Lowest misclassification error across datasets

In general, we see the same pattern as with the Gisette dataset. The training error is almost always lower than the validation error.

3. On the Gisette dataset, for each off $k \in \{3, 10, 30, 100, 300\}$ train a random forest with k trees where the split attribute at each node is chosen from a random subset of $71 \approx \sqrt{5000}$ features. Use the trained trees to predict the class labels on the training and test sets and obtain the training and test misclassification errors. Plot on the same graph the training and test errors vs. number of trees k as two separate curves. Report the training and test misclassification errors in a table.

We do this in Python, and obtain the plot of the training and test misclassification errors vs tree depth shown in figure 3:

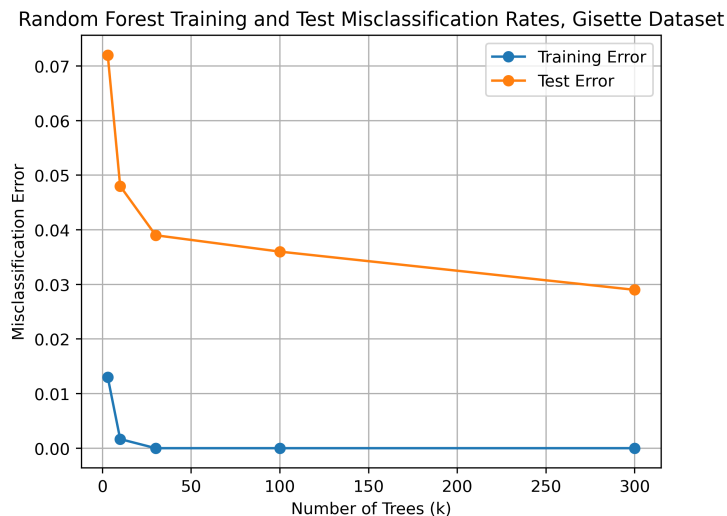


Figure 3: Misclassification Error vs Tree Depth, Training and Validation: Gisette dataset

The table reporting the training and test errors for various random forests are shown in table 3:

Number of Trees	Training Misclassification Error	Validation Misclassification Error
3	0.013000	0.072000
10	0.001667	0.048000
30	0.000000	0.039000
100	0.000000	0.036000
300	0.000000	0.029000

Table 3: Misclassification errors, Random Forests, Gisette Dataset

Beyond a certain point, the error rates for both the training and the validation datasets do not improve. This also shows the training error being 0 for 100 and 300 trees in the random forest.

4. Repeat point 3.) on the satimage, madelon, and hill-valley datasets.

The images showing the training and test misclassification errors are displayed in figure 4 and we can create a table for this, shown in table 4.

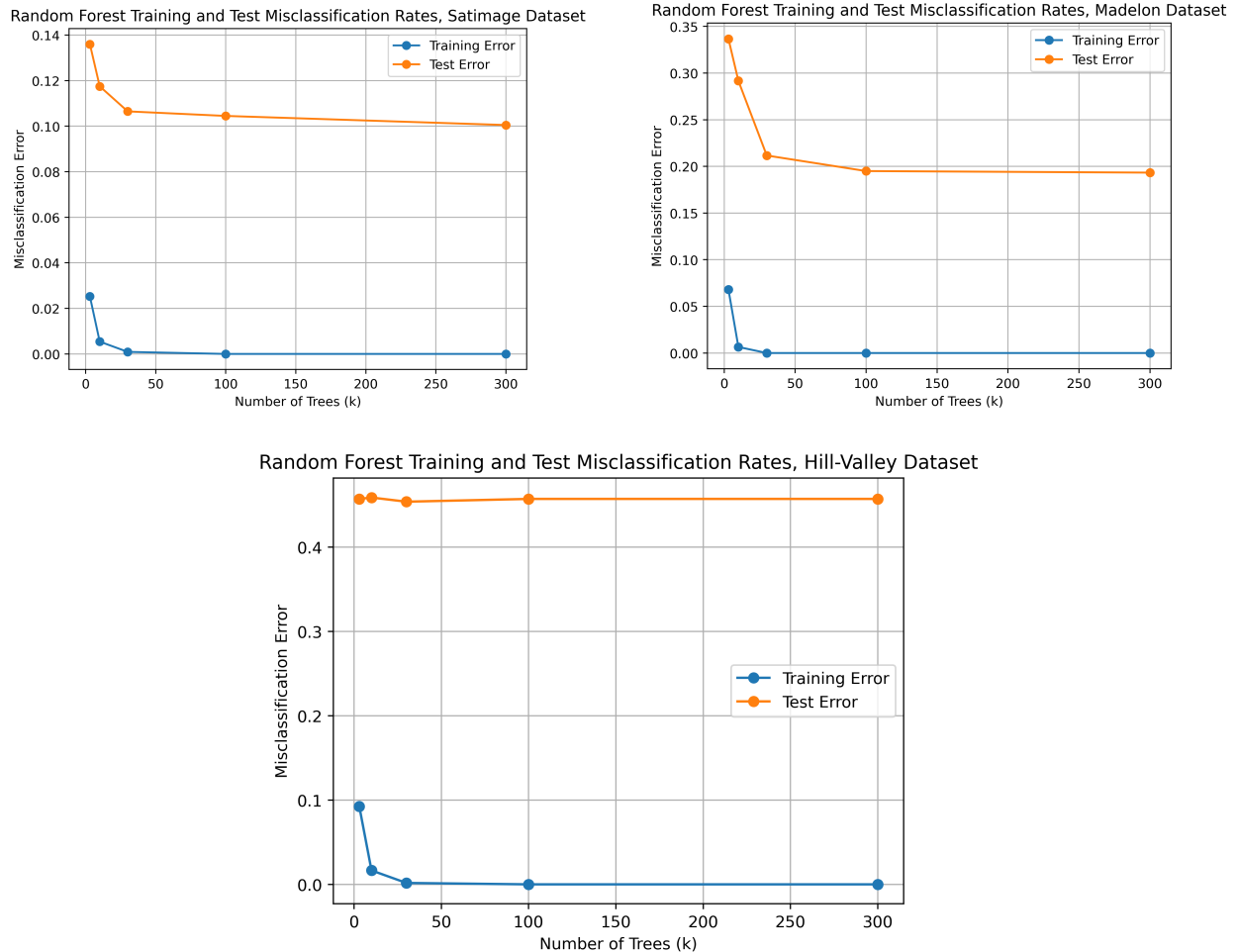


Figure 4: Plots showing training and test misclassification errors versus number of trees.

5. Repeat point 3.) on the Gisette dataset where the split attribute at each node is chosen from a random subset of $12 \approx \log_2(5000)$ features, and where the split attribute at each node is chosen from all 5000 features, plotting all four curves (2 train and 2 test misclassification errors) on the same graph.

Doing this gives us figure 5:

In this plot, we can see that the training error is not meaningfully different when we increase the number of features used. In fact, the validation error is worse for the random forest with 5000 features than for the forest with 12 features.

Dataset	Num. Trees	Training Misclass. Err.	Validation Misclass. Err.
Satimage	3	0.025254	0.136000
Satimage	10	0.005411	0.117500
Satimage	30	0.000902	0.106500
Satimage	100	0.000000	0.104500
Satimage	300	0.000000	0.100500
Madelon	3	0.068000	0.336667
Madelon	10	0.006500	0.291667
Madelon	30	0.000000	0.211667
Madelon	100	0.000000	0.195000
Madelon	300	0.000000	0.193333
Hill-Valley	3	0.092409	0.457096
Hill-Valley	10	0.016502	0.458746
Hill-Valley	30	0.001650	0.453795
Hill-Valley	100	0.000000	0.457096
Hill-Valley	300	0.000000	0.457096

Table 4: Misclassification errors for Random Forests across datasets

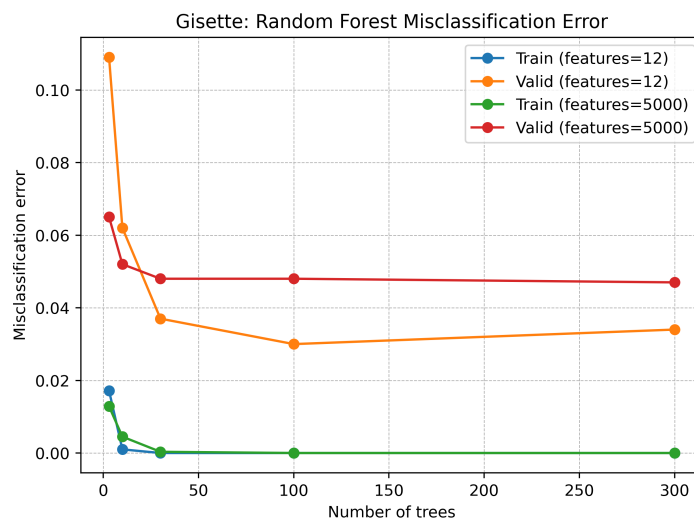


Figure 5: Misclassification Error vs Number of Trees and Number of Features, Training and Validation: Gisette dataset

Code

```

1 import pandas as pd
2 import numpy as np
3 from sklearn import tree
4 from sklearn.metrics import accuracy_score
5 from sklearn.ensemble import RandomForestClassifier
6 import matplotlib.pyplot as plt
7
8 #need a double loop. Outer loop over datasets, inner loop over depts
9 depths = range(1, 13)
10
11 #creating a dict to help loop over the datasets more
12 #got tired of typing the same code for everything
13 #each dict has files paths and some data for working with the results later
14 datasets = [
15     {
16         "name": "Gisette",
17         "X_train": "assignments/hw01/data/gisette_train.data",
18         "y_train": "assignments/hw01/data/gisette_train.labels",
19         "X_valid": "assignments/hw01/data/gisette_valid.data",
20         "y_valid": "assignments/hw01/data/gisette_valid.labels",
21         "fig_out":
22         "assignments/hw01/figures/gisette_misclassification_error.png",
23         "fig_out_2":
24         "assignments/hw01/figures/gisette_misclassification_error_2.png",
25         "latex_caption": "Lowest_misclassification_error, Gisette_Dataset",
26         "latex_caption_2": "Misclassification_errors, Random_Forests, Gisette_
27         Dataset",
28         "latex_label": "tab:gisette_tree_misclass_error"
29     },
30     {
31         "name": "Satimage",
32         "X_train": "assignments/hw01/data/X_sat.dat",
33         "y_train": "assignments/hw01/data/Y_sat.dat",
34         "X_valid": "assignments/hw01/data/Xtest_sat.dat",
35         "y_valid": "assignments/hw01/data/Ytest_sat.dat",
36         "fig_out":
37         "assignments/hw01/figures/satimage_misclassification_error.png",
38         "fig_out_2":
39         "assignments/hw01/figures/satimage_misclassification_error_2.png",
40         "latex_caption": "Lowest_misclassification_error: Satimage_Dataset",
41         "latex_caption_2": "Misclassification_errors, Random_Forests, Satimage_
42         Dataset",
43         "latex_label": "tab:Satimage_tree_misclass_error"
44     },
45     {

```

```

40     "name": "Madelon",
41     "X_train": "assignments/hw01/data/madelon_train.data",
42     "y_train": "assignments/hw01/data/madelon_train.labels",
43     "X_valid": "assignments/hw01/data/madelon_valid.data",
44     "y_valid": "assignments/hw01/data/madelon_valid.labels",
45     "fig_out":
"assignments/hw01/figures/madelon_misclassification_error.png",
46     "fig_out_2":
"assignments/hw01/figures/madelon_misclassification_error_2.png",
47     "latex_caption": "Lowest_misclassification_error:_Madelon_Dataset",
48     "latex_caption_2": "Misclassification_errors,_Random_Forests,_Madelon_
Dataset",
49     "latex_label": "tab:Madelon_tree_misclass_error"
50 },
51 {
52     "name": "Hill-Valley",
53     "X_train": "assignments/hw01/data/X_hv.dat",
54     "y_train": "assignments/hw01/data/Y_hv.dat",
55     "X_valid": "assignments/hw01/data/Xtest_hv.dat",
56     "y_valid": "assignments/hw01/data/Ytest_hv.dat",
57     "fig_out":
"assignments/hw01/figures/hill_valley_misclassification_error.png",
58     "fig_out_2":
"assignments/hw01/figures/hill_valley_misclassification_error_2.png",
59     "latex_caption": "Lowest_misclassification_error:_Hill-Valley_Dataset",
60     "latex_caption_2": "Misclassification_errors,_Random_Forests,_
Hill-Valley_Dataset",
61     "latex_label": "tab:hill_valley_tree_misclass_error"
62 },
63 {
64     "name": "Covtype",
65     "X_train": "assignments/hw01/data/covtype.data",
66     "fig_out":
"assignments/hw01/figures/Covtype_misclassification_error.png",
67     "latex_caption": "Lowest_misclassification_error:_Covtype_Dataset",
68     "latex_caption_2": "Misclassification_errors,_Random_Forests,_Covtype_
Dataset",
69     "latex_label": "tab:Covtype_tree_misclass_error"
70 }
71 ]
72
73 #parts a and b
74 for i in datasets:
75     #special case for the Covtype dataset. Last column is the label
76     if i["name"] == "Covtype":
77         data = pd.read_csv(i["X_train"], header=None).to_numpy()
78

```

```

79     #all columns except the last one
80     X = data[:, :-1]
81     y = data[:, -1]
82
83     #the first 11340 + 3780 rows are training
84     X_train = X[:15120]
85     y_train = y[:15120]
86
87     #the rest test
88     X_valid = X[15120:]
89     y_valid = y[15120:]
90 else:
91     #reading in dataset, if dataset is not covtype
92     X_train = pd.read_csv(i["X_train"], delim_whitespace=True, header=None)
93     X_valid = pd.read_csv(i["X_valid"], delim_whitespace=True, header=None)
94
95     y_train = pd.read_csv(i["y_train"], header=None)
96     y_valid = pd.read_csv(i["y_valid"], header=None)
97
98     #basically, we end up with X and y train and X and Y valid
99
100    #initialize empty arrays
101    train_err = []
102    valid_err = []
103
104    #inner loop looping over the tree depths
105    for d in depths:
106        #fit tree
107        dec_tree = tree.DecisionTreeClassifier(max_depth=d)
108        dec_tree = dec_tree.fit(X_train, y_train)
109
110        #make predictions
111        training_predictions = dec_tree.predict(X_train)
112        validation_predictions = dec_tree.predict(X_valid)
113
114        #compute error
115        training_error = 1 - accuracy_score(y_train, training_predictions)
116        validation_error = 1 - accuracy_score(y_valid, validation_predictions)
117
118        train_err.append(training_error)
119        valid_err.append(validation_error)
120
121    #now need to plot this
122    plt.figure()
123    plt.plot(list(depths), train_err, marker="o", label="Training_error")
124    plt.plot(list(depths), valid_err, marker="o", label="Validation_error")
125    plt.title(f"Training and Test Misclassification Rates, {i['name']} Dataset")

```

```
126 plt.xlabel("Maximum_depth")
127 plt.ylabel("Misclassification_error")
128 plt.legend()
129 plt.grid(True, alpha=0.3)
130 plt.savefig(i["fig_out"], dpi=400, bbox_inches="tight")
131 plt.close()
132
133 #need a table to find the best depth
134 results = pd.DataFrame({
135     "Depth": list(depths),
136     "Training_Misclassification": train_err,
137     "Validation_Misclassification": valid_err
138 })
139
140 #find lowest validation error and tree depth
141 lowest_error = results[results['Validation_Misclassification'] ==
142 results['Validation_Misclassification']].min()
143
144 #save as latex table:
145 LATEX_table = lowest_error.to_latex(
146     index=False,
147     caption=i["latex_caption"],
148     label=i["latex_label"],
149     column_format="ccc"
150 )
151
152 #print
153 print(LATEX_table)
154
155 #parts c and d
156 k_values = [3, 10, 30, 100, 300]
157
158 #only need some of the datasets in the dict
159 selected_names = {"Gisette", "Satimage", "Madelon", "Hill-Valley"}
160 filtered_datasets = list(filter(lambda da: da["name"] in selected_names,
161     datasets))
162
163 #loop over filter datasets for parts c and d:
164 for data in filtered_datasets:
165     print(data["name"])
166
167 #get data
168 X_train = pd.read_csv(data["X_train"], delim_whitespace=True, header=None)
169 X_valid = pd.read_csv(data["X_valid"], delim_whitespace=True, header=None)
170
171 #got warning about y being a column vector. Python recommended using ravel
```

```
171 y_train = pd.read_csv(data["y_train"], header=None).values.ravel()
172 y_valid = pd.read_csv(data["y_valid"], header=None).values.ravel()
173
174 train_errors = []
175 test_errors = []
176
177 #iterate over k values
178 for k in k_values:
179     forest = RandomForestClassifier(n_estimators=k, max_features=71)
180     forest = forest.fit(X_train, y_train)
181
182     #making predictions
183     prediction_training = forest.predict(X_train)
184     prediction_validation = forest.predict(X_valid)
185
186     #error
187     train_error = 1 - accuracy_score(y_train, prediction_training)
188     valid_error = 1 - accuracy_score(y_valid, prediction_validation)
189
190     #append to array
191     train_errors.append(train_error)
192     test_errors.append(valid_error)
193
194 #need to plot this stuff now
195 plt.figure()
196 plt.plot(k_values, train_errors, marker='o', label='Training_Error')
197 plt.plot(k_values, test_errors, marker='o', label='Test_Error')
198
199 plt.xlabel("Number_of_Trees_(k)")
200 plt.ylabel("Misclassification_Error")
201 plt.title(f"Random_Forest_Training_and_Test_Misclassification_Rates,_{data['name']} Dataset")
202 plt.legend()
203 plt.grid(True)
204 plt.savefig(data["fig_out_2"], dpi=400, bbox_inches="tight")
205 plt.close()
206
207 #table of errors
208 results = pd.DataFrame({
209     "Number_of_Trees": k_values,
210     "Training_Misclassification_Error": train_errors,
211     "Validation_Misclassification_Error": test_errors
212 })
213
214 #need a latex table
215 LATEX_table = results.to_latex(
216     index=False,
```

```
217     caption=data["latex_caption_2"],
218     column_format="ccc"
219 )
220
221     #print
222     print(LATEX_table)
223
224 #part e
225 #need two parts, one where each node is choen from a random subset of 12
226 #and another where each node is chosen from all 5000 features.
227 #None means use all features
228 node_amount = [12, None]
229
230 results = []
231
232 #need to get the Gisette dataset
233 for i in datasets:
234     if i["name"] == "Gisette":
235         X_train = pd.read_csv(i["X_train"], delim_whitespace=True, header=None)
236         X_valid = pd.read_csv(i["X_valid"], delim_whitespace=True, header=None)
237
238         y_train = pd.read_csv(i["y_train"], header=None).values.ravel()
239         y_valid = pd.read_csv(i["y_valid"], header=None).values.ravel()
240
241         break
242
243 for features in node_amount:
244     print(features)
245     #cleaning up for later
246     if features == 12:
247         label = "12"
248     else:
249         label = "5000"
250
251     #loop over the number of trees
252     for k in k_values:
253         print(k)
254         randomfor = RandomForestClassifier(n_estimators=k, max_features=features)
255         randomfor = randomfor.fit(X_train, y_train)
256
257         prediction_training = randomfor.predict(X_train)
258         prediction_validation = randomfor.predict(X_valid)
259
260         #error
261         train_error = 1 - accuracy_score(y_train, prediction_training)
262         valid_error = 1 - accuracy_score(y_valid, prediction_validation)
263
```

```
264         results.append({
265             "features": label,
266             "k": k,
267             "train_err": train_error,
268             "valid_err": valid_error
269         })
270
271 #creating dataframe from the list of results
272 df = pd.DataFrame(results)
273
274 plt.figure()
275
276 #needed to convert to numpy for some reason?
277 for label in ["12", "5000"]:
278     sub = df[df["features"] == label].sort_values("k")
279     plt.plot(sub["k"].to_numpy(), sub["train_err"].to_numpy(),
280             marker="o", label=f"Train_{(features={label})}")
281     plt.plot(sub["k"].to_numpy(), sub["valid_err"].to_numpy(),
282             marker="o", label=f"Valid_{(features={label})}")
283
284 plt.xlabel("Number_of_trees")
285 plt.ylabel("Misclassification_error")
286 plt.title("Gisette:_Random_Forest_Misclassification_Error")
287 plt.grid(True, which="both", linestyle="--", linewidth=0.5)
288 plt.legend()
289 plt.tight_layout()
290 plt.savefig("assignments/hw01/figures/gisette_misclassification_error_3.png",
291           dpi=400, bbox_inches="tight")
291 plt.close()
```